

计算机科学与技术学院
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

软件逆向工程

第六讲 KEYFILE等保护方式的逆向和破解实验

软件版权的技术保护

- ◆ 软件版权的技术保护目标及基本原则
- ◆ 软件版权保护基本技术

软件版权的技术保护目标及基本原则：

软件版权保护的目标

- 1 防软件盗版，即对软件进行防非法复制和使用的保护。
- 2 防逆向工程，即防止软件被非法修改或剽窃软件设计思想等。
- 3 防信息泄露，即对软件载体及设计数据的保护，如加密硬件、加密算法密钥等。

软件版权的技术保护目标及基本原则：

软件版权保护的基本原则

- 1 实用和便利性。
- 2 可重复使用。
- 3 有限的交流和分享。

软件版权保护基本技术

基于硬件的保护技术

- 1 对发行介质的保护。
- 2 软件狗。
- 3 可信计算芯片。

软件版权保护基本技术

基于硬件的保护技术

1 对发行介质的保护。

光盘软件保护

为了能有效地防止光盘盗版，从技术上来说要解决三个问题：

- (1) 要防止光盘之间的拷贝；
- (2) 要防止破解和跟踪加密光盘；
- (3) 要防止光盘与硬盘的拷贝。

目前防止光盘盗版的技术有：

1.特征码技术

特征码技术是通过是光盘上的特征码，如SID (Source Ident-idification Code)来区分是正版还是盗版光盘。

该特征码是在光盘压制生产时自然产生的，而不同的母盘压制的特征码不一样。光盘上的软件运行时必须先使用该特征码，而这种特征码在盗版者翻制光盘过程中是无法提取和复制的。

软件版权保护基本技术

基于硬件的保护技术

1 对发行介质的保护。

光盘软件保护

2.非正常导入区

光盘的导入区TOC (Track On CD)是用来记录有关光盘类型等信息，是由光盘自动产生的，并且光盘无法复制非正常的导入区。因此，在导入区内添加重要数据以供读盘使用，便能有效地防止光盘之间的非法复制。

3. 非正常扇区、修改文档结构

软件版权保护基本技术

基于硬件的保护技术

2 软件狗。

- 软件狗 (dongles) 又称加密锁等，是一个可安装在计算机并口、串口或USB接口上的硬件小插件。同时有一套适用于各种语言的接口软件和工具软件。

当被软件狗保护的应用程序运行时，程序向插在计算机上的软件狗发出查询命令，软件狗迅速计算查询并给出响应。如果响应正确，软件将继续运行，否则程序将停止工作。

- 软件狗技术属于硬加密技术，软件狗中单片机里包含有专用于加密算法软件，不能再被读取出来。这样，就保证了软件狗具有硬件不可被复制、加密强度大、可靠性高等特点，软件狗广泛应用于计算机商业软件保护。

软件版权保护基本技术

基于软件的保护技术

1 注册验证

- 1) 安装序列号方式。
- 2) 用户名+序列号方式。
- 3) 在线激活注册方式。
- 4) 许可证保护方式。

序列号保护机制

软件验证序列号的合法性过程就是验证用户名和序列号之间的换算关系，即数学映射关系是否正确过程。

- 1.以用户名生成序列号

序列号=F(用户名)

- 2.通过注册码来验证用户名的正确性

序列号=F(用户名)

用户名=F(序列号)

序列号保护机制

软件验证序列号的合法性过程就是验证用户名和序列号之间的换算关系，即数学映射关系是否正确过程。

- 3.通过对等函数检查注册码

$F1(\text{用户名}) = F2(\text{序列号})$

- 4.同时采用用户名和序列号作为自变量

特征值 = $F(\text{用户名}, \text{序列号})$

特征值 = $F(\text{用户名}1, \text{用户名}2, \dots, \text{序列号}1, \text{序列号}2, \dots)$

攻击序列号保护

利用数据约束性（只对用明文进行比较注册码有效）

大多数使用明文进行注册码比较真伪过程中，真正的注册码会在某个时刻出现在内存中，大多数情况下，真正的注册码会在用户输入的注册码的内存地址-90h~+90h字节的地方

利用消息断点

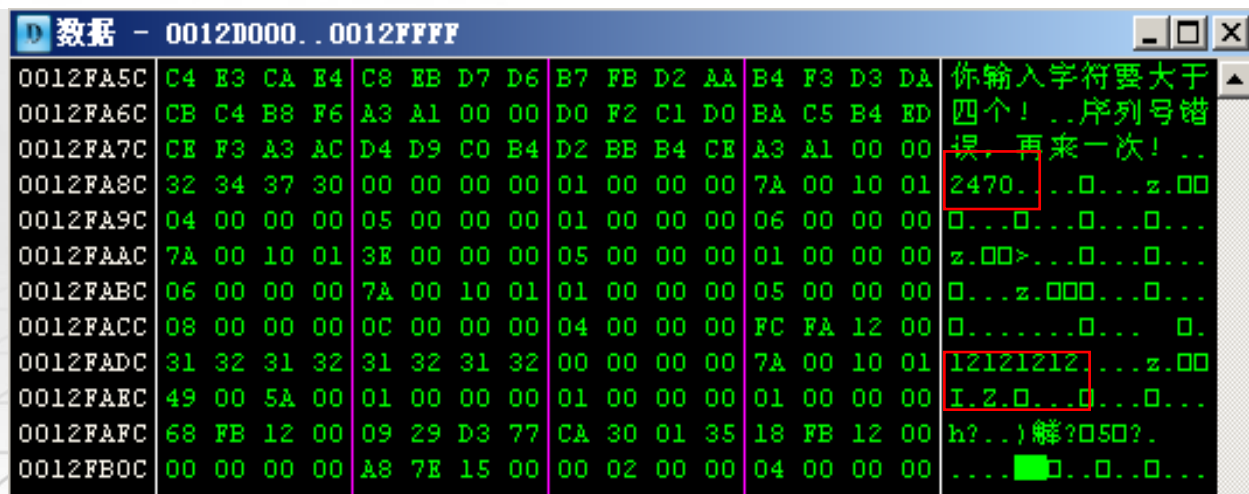
许多序列号验证时都存在一个验证按钮，当点击按钮时，将发送鼠标消息，利用此消息能够找到按钮对应的事件代码

利用提示信息

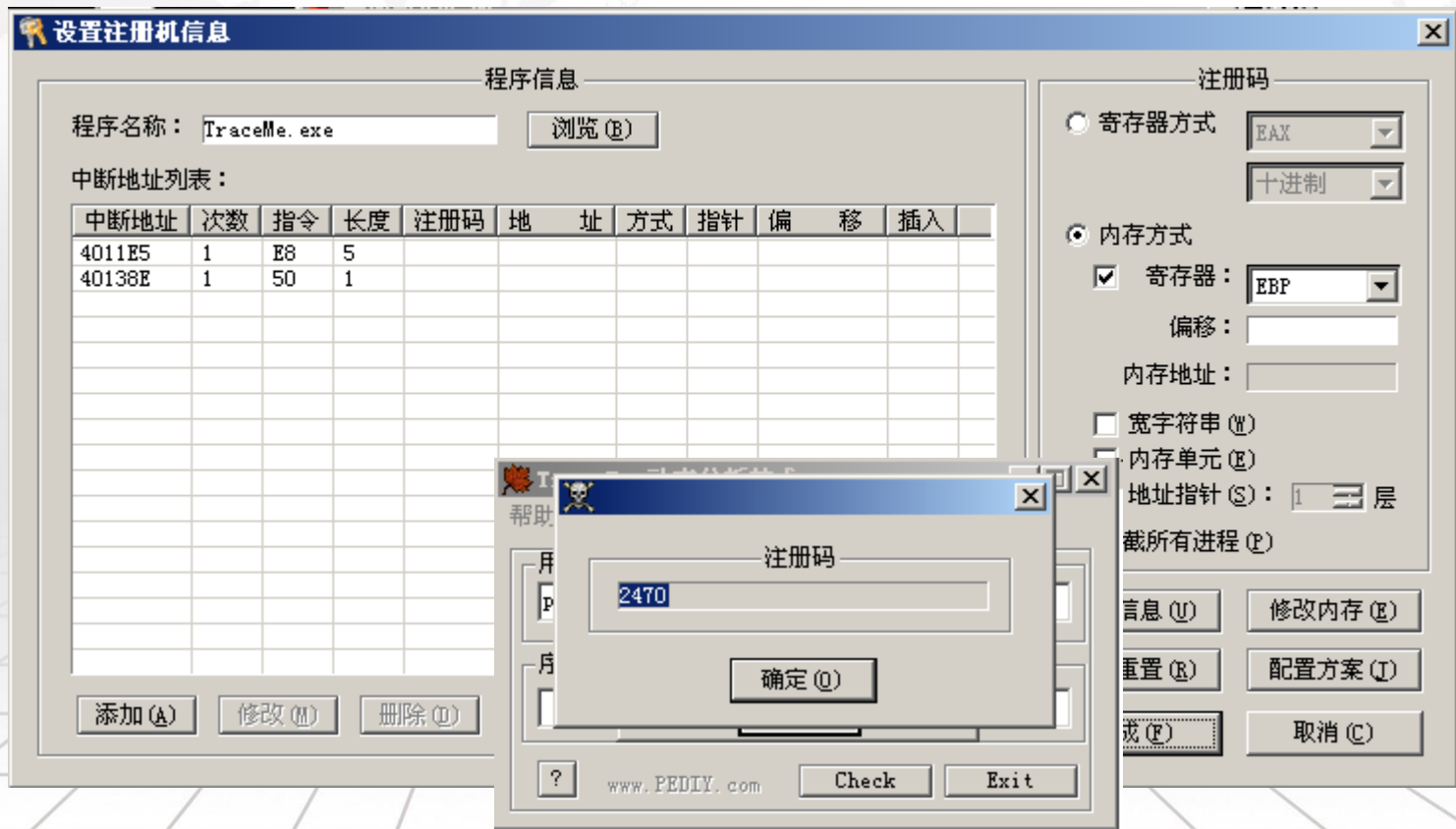
利用输入错误的提示信息，来查找对应的验证代码处

例1.TraceMe

OllyDbg 也可以实现这种查找功能。用 OllyDbg 加载 TraceMe, 输入假序列号, 单击 “Check” 按钮直到出现错误提示框。按 “Alt+M” 快捷键打开内存窗口, 在上面一行按 “Ctrl+B” 快捷键打开搜索框, 搜索刚输入的序列号 “12121212”, 如图 5.2 所示。OllyDbg 的数据查找功能非常有用, 可以在当前进程的整个内存映像里查找数据。



例2.Keymake



例3.Serial.exe

00401228	push	0040218E	;字符串“pediy”入栈, 设其为 Name
0040122D	call	0040137E	;计算 k1, k1=F1(用户名)
00401232	push	eax	;k1 入栈(通过栈传递变量)
00401233	push	0040217E	;字串“1234”入栈, 设其为 Code
00401238	call	004013D8	;计算 k2, k2=F2(序列号)
0040123D	add	esp, 4	;平衡栈, 上面 call 指令调用的 k2 是通过 ebx 传出的
00401240	pop	eax	;k1 出栈
00401241	cmp	eax, ebx	;比较 k1 和 k2, 即 F1(用户名)=F2(序列号)
00401243	je	short 0040124C	;如果相等, 注册成功
00401245	call	00401362	;如果不相等, 注册失败

软件版权保护基本技术

基于软件的保护技术

3 代码混淆

- 1) 代码混淆 (Code Obfuscation) 技术也称为代码迷惑技术。通过代码混淆技术可以将源代码转换为与之功能上等价，但是逆向分析难度增大了目标代码，这样即使逆向分析人员反编译了源程序，也难以得到源代码所采用的的算法、数据结构等关键信息。因此，代码混淆可以抵御逆向工程、代码篡改等攻击行为。
- 2) 代码混淆按照保护方式的侧重点不同可分为4类：布局混淆、控制流混淆、数据混淆和预防性混淆。

软件版权保护基本技术

4 软件加壳

- 1) 加壳是指，在原二进制文件（如可执行文件、动态链接库）上附加一段专门负责保护该文件不被反编译或非法修改的代码或数据，以上源文件进行加密或压缩，并修改原文件的运行参数或运行流程，使其被加载到内存中执行时，附加的这段代码—保护壳，先于原程序运行，执行过程中先对原程序文件进行解密和还原，完成后再将控制权转交给原程序。加壳后的程序能够增加逆向（静态）分析和非法修改的难度。
- 2) 根据对原程序实施保护方式的不同，壳大致可以分为两类：压缩保护型壳和加密保护型壳。

软件版权保护基本技术

基于软件的保护技术

5 虚拟机保护

- 1) 原理是，先模拟产生自己定制的虚拟机，然后将软件程序集代码为这个模拟产生的虚拟机才能解释执行的虚拟机代码。
- 2) 由于软件执行的时候部分运算是在虚拟机中进行的，虚拟机的复杂度很高，软件攻击者需要了解虚拟机的结构或者看懂虚拟机指令集才能逆向成功，这无疑加大了软件程序集代码被逆向的难度，极大提高了软件程序集的保护强度。

5.2警告窗口

去除警告窗口常用的3种方法是修改程序的资源、静态分析及动态分析。

- ①将可执行文件中警告窗口的属性设置为透明或不可见
- ②找到创建该窗口的代码进行跳过
- ③利用消息断点设置断点拦截

```
int DialogBoxParam(  
    HINSTANCE hInstance,           //应用程序实例句柄  
    LPCTSTR lpTemplateName,        //对话框 ID  
    HWND hWndParent,               //父窗口句柄  
    DLGPROC lpDialogFunc,          //对话框处理函数指针  
    LPARAM dwInitParam             //初始化值  
);
```

跳过警告窗口代码。将“00401051 push 00000000”改成“00401051 jmp 4010E5”。

5.2警告窗口

去除警告窗口常用的3种方法是修改程序的资源、静态分析及动态分析。

- ①将可执行文件中警告窗口的属性设置为透明或不可见
- ②找到创建该窗口的代码进行跳过
- ③利用消息断点设置断点拦截

```
:00401051    push 00000000
:00401053    push 00401109          ;将此处指向主窗口的子处理程序
:00401058    push 00000000
:0040105A    push 00000065          ;指向主对话框的 ID DialogID_0065
:0040105C    push eax
:0040105D    mov dword ptr [0040119C], eax
* Reference To: USER32.DialogBoxParamA, Ord:0093h
:00401062    Call dword ptr [00401010] ;该函数会调用主对话框窗口
:00401068    xor eax, eax
```

5.3时间限制

时间限制的两种方式：

①限制每次运行的时长

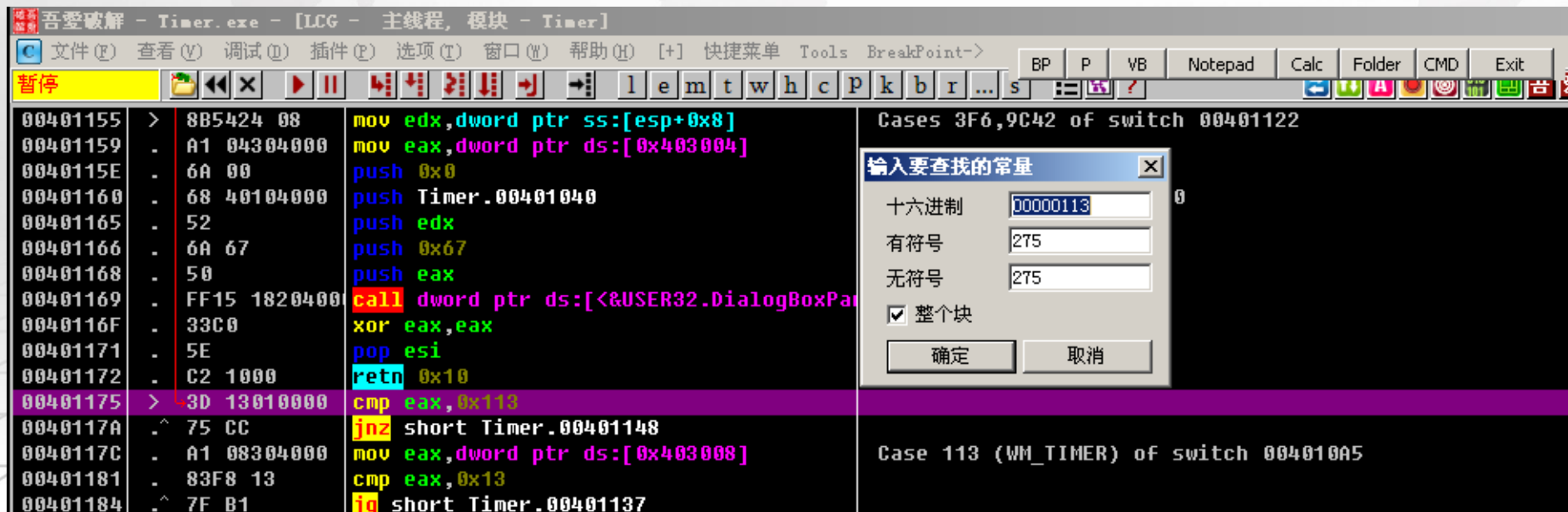
②每次运行的时长不限，但有总共的时间限制

程序可用的定时器函数：SetTimer、timeSetEvent、GetTickCount、timeGetTime

程序可用的获取时间函数：GetSystemTime、GetLocalTime、GetFileTime或者读取系统需要频繁修改的文件（user.dat、system.dat等）的最后修改日期，然后调用FileTimeToSystemTime函数转换为系统日期格式，确定当前系统日期

5.3 破解时间限制

1. 直接跳过SetTimer函数，不产生WM_TIMER消息
2. 通过查找WM_TIMER消息对应的消息码0x113，定位到设置WM_TIMER处，修改消息就能取消时间限制了



5.4 去除菜单功能限制

1. 正式版本将所有菜单全部打开，但试用版本将部分菜单隐藏置灰

2. 菜单或窗口置灰不可用函数：
EnableMenuItem、EnableWindow

破解方法：

断点让菜单或窗口置灰不可用函数，修改传入的代表不可用或置灰的参数，修改为正常



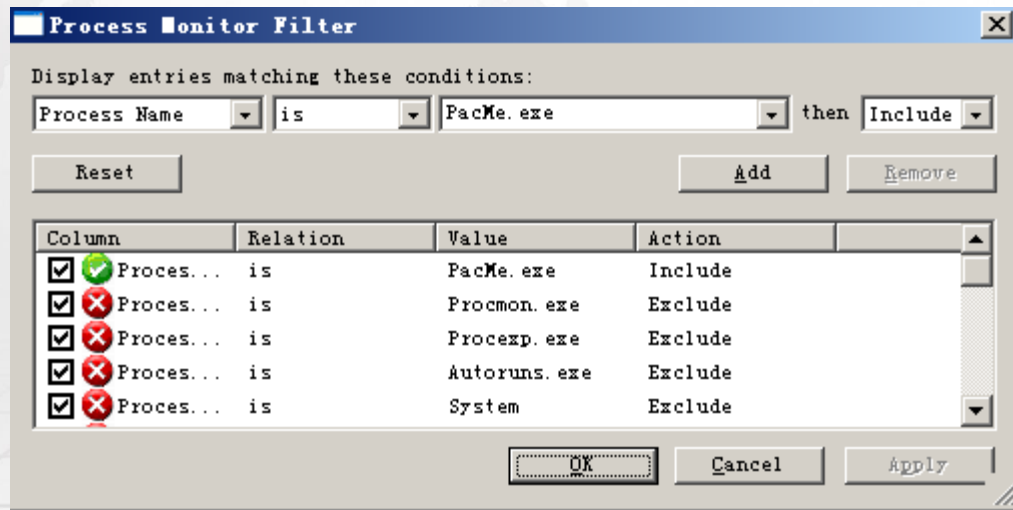
5.5 Keyfile文件保护

与KeyFile相关的API函数

API函数	用于注册文件时的主要作用
FindFirstFileA	确定注册文件是否存在
CreateFileA,_lopen	确定文件是否存在，打开文件获取其句柄
GetFileSize,GetFileSizeEx	确定文件大小
GetFileAttributesA,GetFileAttributesExA	获得注册文件属性
SetFilePointer,SetFilePointerEx	移动文件指针
ReadFile	读取文件内容

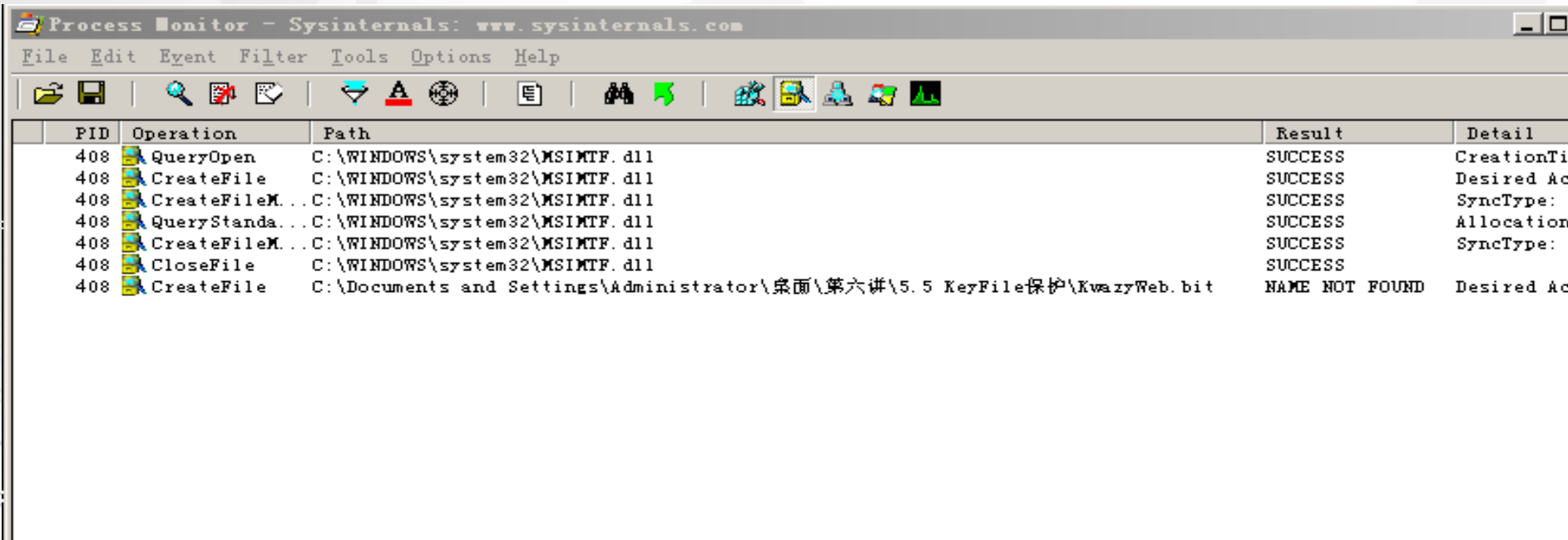
5.5 Keyfile文件保护

1.使用文件监视工具Process Monitor，单击菜单项“filter”，设置过滤条件。



5.5 Keyfile文件保护

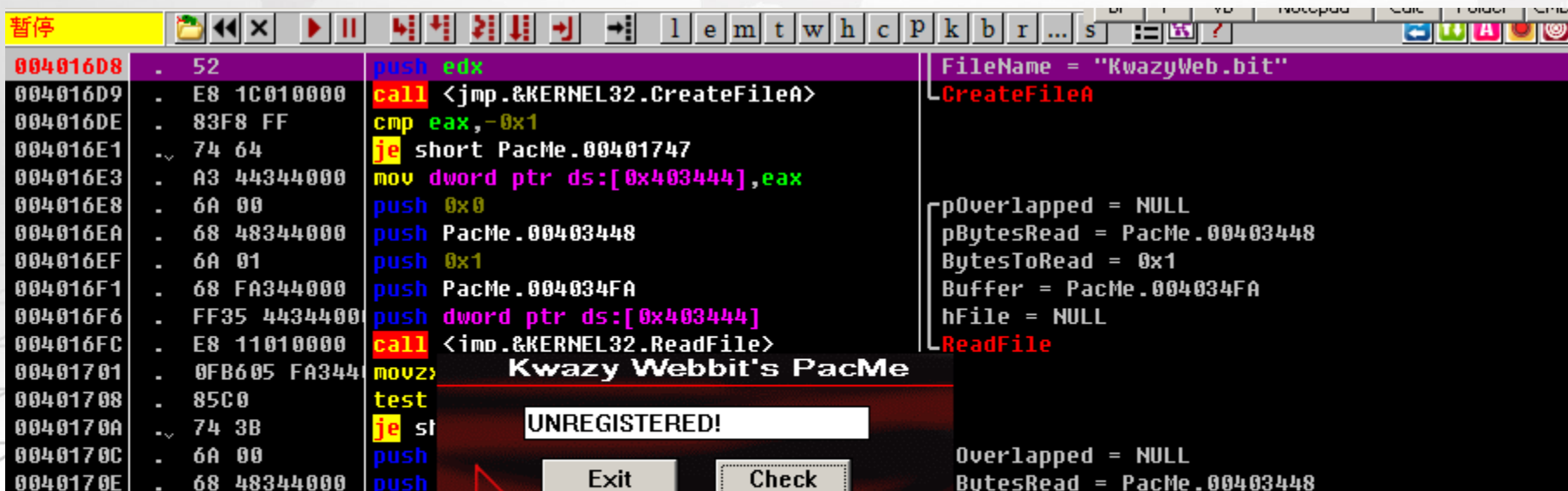
2. 打开PacMe.exe，使用文件监视工具Process Monitor监视文件获得KeyFile的文件名。



5.5 Keyfile文件保护

2

除了用 Process Monitor 监视文件获得 KeyFile 文件名，也可以直接对文件的相关函数设断，从而获得 KeyFile 的相关信息。用 OllyDbg 装载 PacMe 后，按“F9”键运行 PacMe。用 CreateFileA 函数设断，单击 PacMe 的“Check”按钮，中断代码如下。



The screenshot shows the OllyDbg interface with the assembly window displaying code for 'Kwazy Webbit's PacMe'. The code includes instructions like `push edx`, `call <jmp.&KERNEL32.CreateFileA>`, `cmp eax,-0x1`, `je short PacMe.00401747`, `mov dword ptr ds:[0x403444],eax`, `push 0x0`, `push PacMe.00403448`, `push 0x1`, `push PacMe.004034FA`, `push dword ptr ds:[0x403444]`, `call <inp.&KERNEL32.ReadFile>`, `movz`, `test`, `je st`, and `push`. The function call log on the right shows the parameters for `CreateFileA`: `FileName = "KwazyWeb.bit"`, `pOverlapped = NULL`, `pBytesRead = PacMe.00403448`, `BytesToRead = 0x1`, `Buffer = PacMe.004034FA`, and `hFile = NULL`. The bottom of the window shows a dialog box with the text 'UNREGISTERED!' and buttons for 'Exit' and 'Check'.

```
004016D8 . 52          push     edx
004016D9 . E8 1C010000 call     <jmp.&KERNEL32.CreateFileA>
004016DE . 83F8 FF     cmp     eax,-0x1
004016E1 ~ 74 64       je      short PacMe.00401747
004016E3 . A3 44344000 mov     dword ptr ds:[0x403444],eax
004016E8 . 6A 00       push    0x0
004016EA . 68 48344000 push    PacMe.00403448
004016EF . 6A 01       push    0x1
004016F1 . 68 FA344000 push    PacMe.004034FA
004016F6 . FF35 44344000 push    dword ptr ds:[0x403444]
004016FC . E8 11010000 call     <inp.&KERNEL32.ReadFile>
00401701 . 0FB605 FA344 movz
00401708 . 85C0       test
0040170A ~ 74 3B       je      st
0040170C . 6A 00       push
0040170E . 68 48344000 push
```

FileName = "KwazyWeb.bit"
CreateFileA
pOverlapped = NULL
pBytesRead = PacMe.00403448
BytesToRead = 0x1
Buffer = PacMe.004034FA
hFile = NULL
ReadFile

UNREGISTERED!
Exit Check

5.5 Keyfile文件保护

OllyDbg 会直接把 CreateFileA 函数读取的文件名显示出来。KeyFile 名为“KwazyWeb.bit”。用十六进制工具伪造一个 KeyFile，建议将其内容设置为一些有规律的数字，例如 1、2、3、4、5……以便在跟踪时进行分析。

004016D8	- 52	push edx	FileName = "腋?"
004016D9	- E8 1C010000	call <jmp.&KERNEL32.CreateFileA>	CreateFileA
004016DE	- 83F8 FF	cmp eax,-0x1	
004016E1	- 74 64	je short PacMe.00401747	
004016E3	- A3 44344000	mov dword ptr ds:[0x403444],eax	
004016E8	- 6A 00	push 0x0	pOverlapped = NULL
004016EA	- 68 48344000	push PacMe.00403448	pBytesRead = PacMe.00403448
004016EF	- 6A 01	push 0x1	BytesToRead = 1
004016F1	- 68 FA344000	push PacMe.004034FA	Buffer = PacMe.004034FA
004016F6	- FF35 44344000	push dword ptr ds:[0x403444]	hFile = 0000008C (
004016FC	- E8 11010000	call <jmp.&KERNEL32.ReadFile>	ReadFile
00401701	- 0FB605 FA344	movzx eax,byte ptr ds:[0x4034FA]	
00401708	- 85C0	test eax,eax	
0040170A	- 74 3B	je short PacMe.00401747	
0040170C	- 6A 00	push 0x0	pOverlapped=NULL
0040170E	- 68 48344000	push PacMe.00403448	pBytesRead=PacMe.00403448
00401713	- 50	push eax	BytesToRead (字节大小)
00401714	- 68 88324000	push PacMe.00403288	Buffer=00403288 (缓存地址)
00401719	- FF35 44344000	push dword ptr ds:[0x403444]	hFile=NULL
0040171F	- E8 EE000000	call <jmp.&KERNEL32.ReadFile>	ReadFile
00401724	- E8 D7F8FFFF	call PacMe.00401000	

5.5 Kevfile 文件保护

```
00401724    call    00401000
```

```
{
```

```
    00401000    xor     eax, eax
```

```
    00401002    xor     edx, edx
```

```
    00401004    xor     ecx, ecx
```

```
    00401006    mov     cl, byte ptr [4034FA]
```

```
    0040100C    mov     esi, 00403288
```

```
    00401011    lods    byte ptr [esi]
```

```
    00401012    add     edx, eax
```

```
    00401014    loopd   short 00401011
```

```
    00401016    mov     byte ptr [4034FB], dl
```

```
    0040101C    retn
```

;将刚读取的字节数求和（用户名求和）

;清零

;清零

;清零

;第1个字节，计数器（姓名的长度）

;esi 指向第2个字节后的数据

;将[esi]指向的1字节数据放到 eax 中

;将相加结果放到 edx 中

;循环

;将计算结果放到[4034FB]处

```
00401729    push    0
```

```
0040172B    push    00403448
```

```
00401730    push    12
```

```
00401732    push    004034E8
```

```
00401737    push    dword ptr [403444]
```

```
0040173D    call    <&KERNEL32.ReadFile>
```

```
00401742    call    004010C9
```

```
00401747    push    dword ptr [403444]
```

```
0040174D    call    <&KERNEL32.CloseHandle>
```

;/pOverlapped=NULL

;/pBytesRead=PacMe.00403448

;/BytesToRead=12(18.)

;/Buffer=PacMe.004034E8

;/hFile=NULL

;第3次读取数据，长度为12h(18)

;验证的子程序，计算核心

;/hObject=NULL

;/CloseHandle, 关闭文件

5.5 Keyfile 文件分析

```
1 int sub_4010C9()
2 {
3     int result; // eax@3
4     unsigned __int8 v1; // [sp+2h] [bp-2h]@1
5     char v2; // [sp+3h] [bp-1h]@2
6
7     lstrcpyA(String1, String2);
8     off_403184 = String1 + 16;
9     sub_40101D();
10    v1 = 0;
11 LABEL_2:
12    v2 = 8;
13    while ( 1 )
14    {
15        v2 -= 2;
16        result = sub_401033((byte_4034E8[v1] >> v2) & 3);
17        if ( !result )
18            return result;
19        if ( !v2 )
20        {
21            if ( ++v1 != 18 )
22                goto LABEL_2;
23            return result;
24        }
25    }
26 }
```

000004C9 sub_4010C9:1

大致看一下

5.5 Keyfile

```
1 int sub_4010C9()
2 {
3     int result; // eax@3
4     unsigned __int8 v1; // [sp+2h] [bp-2h]@1
5     char v2; // [sp+3h] [bp-1h]@2
6
7     lstrcpyA(String1, String2);
8     off_403184 = String1 + 16;
9     sub_40101D();
10    v1 = 0;
11 LABEL_2:
12    v2 = 8;
13    while ( 1 )
14    {
15        v2 -= 2;
16        result = sub_401033((byte_4034E8[v1] >> v2) & 3);
17        if ( !result )
18            return result;
19        if ( !v2 )
20        {
21            if ( ++v1 != 18 )
22                goto LABEL_2;
23            return result;
24        }
25    }
26 }
```

000004C9 sub_4010C9:1

5.5 Keyfile 文件保护

```
1 signed int __usercall sub_401033@<eax>(char a1@<a1>)
2 {
3     signed int result; // eax@9
4     _BYTE *v2; // [sp+4h] [bp-4h]@1
5
6     v2 = off_403184;
7     if ( a1 )
8     {
9         if ( a1 == 1 )
10        {
11            off_403184 = (char *)off_403184 + 1;
12        }
13        else if ( a1 == 2 )
14        {
15            off_403184 = (char *)off_403184 + 16;
16        }
17        else
18        {
19            off_403184 = (char *)off_403184 - 1;
20        }
21    }
22    else
23    {
24        off_403184 = (char *)off_403184 - 16;
25    }
```

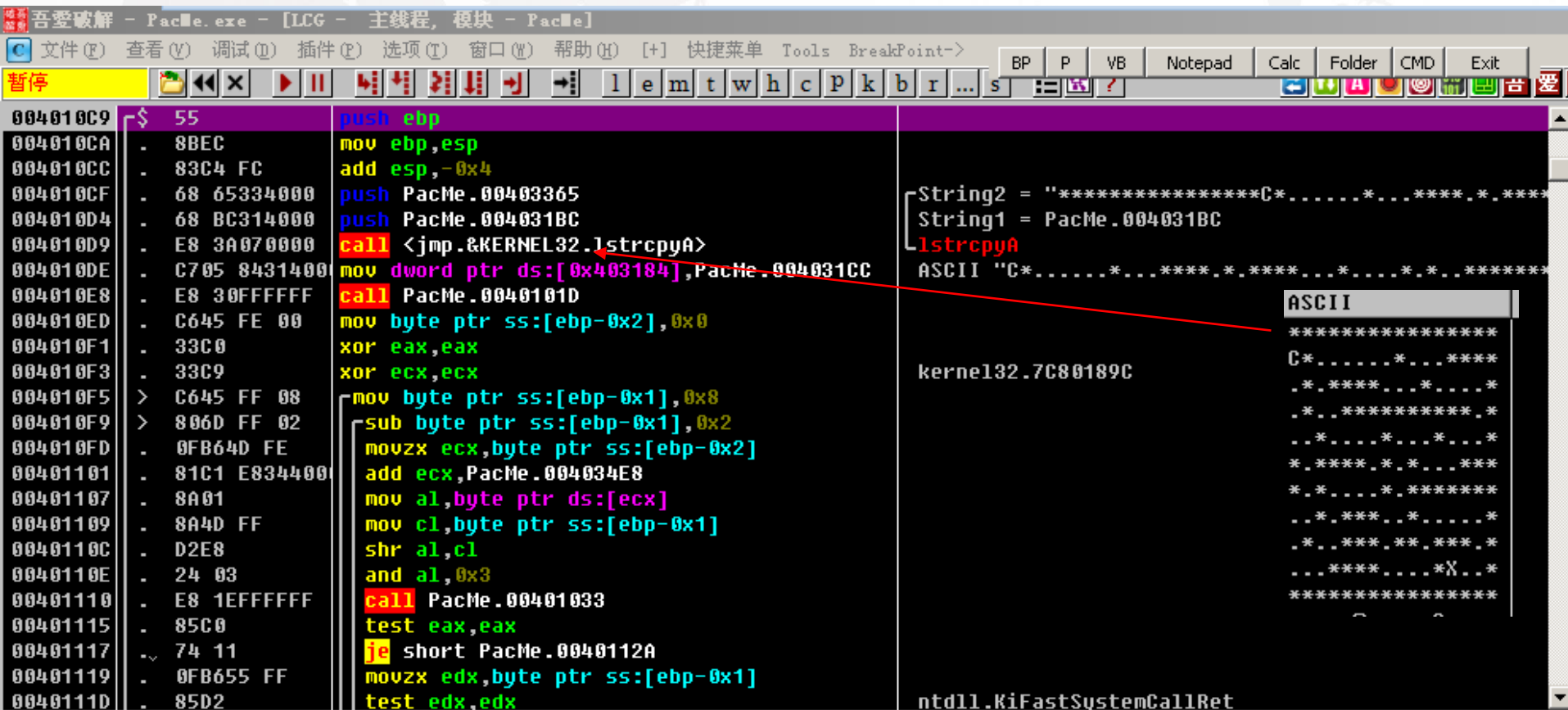
对函数传入的参数a1进行判断，有4条对应的分支

5.5 Keyfile文件保护

```
5  if ( *(_BYTE *)off_403184 == 42 )
7  {
3   result = 0;
}
}
3  else
1  {
2   if ( *(_BYTE *)off_403184 == 88 )
3   {
4       MessageBoxA(0, aCongratulation, aSuccess__, 0);
5       SetWindowTextA((HWND)dword_403420, aCrackedBy);
6   }
7   *(_BYTE *)off_403184 = 67;
3   *v2 = 32;
}   result = 1;
3  }
1  return result;
2 }
```

这里会对之前操作的off_403184进行判断，如果等于88（'X'），则成功。如果等于42（'*'），则返回0。

5.5 Keyfile文件保护



5.5 Keyfile文件保护

0040101D	\$ 8A15 FB344000	mov dl,byte ptr ds:[0x4034FB]
00401023	. B9 12000000	mov ecx,0x12
00401028	. B8 E8344000	mov eax,PacMe.004034E8
0040102D	> 3010	xor byte ptr ds:[eax],dl
0040102F	. 40	inc eax
00401030	. ^ E2 FB	loopd short PacMe.0040102D
00401032	. C3	ret
00401033	\$ 55	push ebp

```

3] ;将第 2 次读取的数据的和放到 d1 中
;ecx 是一个计数器, 值为 12h (18)
;将第 3 次读取的数据指针放到 eax 中
;与 d1 异或
;指向下一个数据
;循环
;设 Strxor 指向异或后的 004034E8h

```

地址	HEX 数据								ASCII																			
004034E8	04	04	04	04	04	04	04	04	04	04	04	04	04															
004034F8	04	04	31	04	00	00	00	00	00	00	00	00	00		1													

```
10 v1 = 0;
11 LABEL_2:
12 v2 = 8;
13 while ( 1 )
14 {
15     v2 -= 2;
16     result = sub_401033((byte_4034E8[v1] >> v2) & 3);
17     if ( !result )
18         return result;
19     if ( !v2 )
20     {
21         if ( ++v1 != 18 )
22             goto LABEL_2;
23         return result;
24     }
25 }
26 }
```

即定义两个循环变量，v2从8开始循环，每次减2（总共4次），参与对18个字符的移位操作，并且与3。作为参数，传递给sub_401033；然后v1++，执行18*4次循环。

对应18个字符中，每个依次取7、6位，5、4位，3、2位，1、0位，然后组成2进制，作为参数传给401033。根据Part1分析：

=0 指针地址减16（C字符上移一行）；

=1 指针地址加1（C字符右移一格）；

=2 指针地址加16（C字符下移一行）；

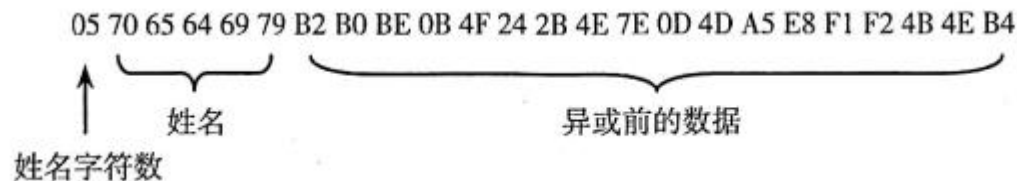
=3 指针地址减1（C字符左移一格）。

这是一个标准的迷宫程序，从“C”开始，一共走 18 次，每次可以走 4 步（18 次大循环和 4 次小循环）。碰到“*”就中断，直到遇见“X”注册成功。路线非常清楚，就是顺着“.”走。按照上面的分析，“0”代表“↑”，“1”代表“→”，“2”代表“↓”，“3”代表“←”，按图一步步前进，就可以得到一系列数据，如图 5.9 所示。

↓↓↓→	↓↓↓←	↓↓→→	↑→↑↑	→→→↑	↑←←←	↑←↑↑	→→→→	→↓→→
2 2 2 1	2 2 2 3	2 2 1 1	0 1 0 0	1 1 1 0	0 3 3 3	0 3 0 0	1 1 1 1	1 2 1 1
↑→→↓	→→→↓	↓←←↓	←←↑←	←↓↓↓	←↓↓→	→→↑↑	→→→→	↓↓←←
0 1 1 2	1 1 1 2	2 3 3 2	3 3 0 3	3 2 2 2	3 2 2 1	1 1 0 0	1 1 1 1	2 2 3 3

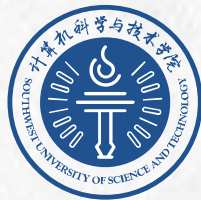
图 5.9 中的数是四进制数，转换成十六进制数为“A9 AB A5 10 54 3F 30 55 65 16 56 BE F3 EA E9 50 55 AF”。然后，程序通过用户名算出一个数，再与上面的十六进制数进行异或运算。

在此以用户名“pediy”推出 KeyFile。“pediy”的十六进制数是“70 65 64 69 79”。KeyFile 由 3 部分组成，如图 5.10 所示。计算步骤如下。



本次实验内容

- 1.完成PacMe的KeyFile算法的逆向分析，画出密钥验证算法流程图；
- 2.以“学号”为用户名，生成Keyfile文件。



计算机科学与技术学院
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

谢谢